

Notebook Checkpoint Walkthrough

Peer Discussion Guide -- 30 Minutes + Q&A

Format

One-on-one peer discussion, informal, scrolling through the notebook together.

30 minutes core walkthrough + 10 minutes for questions.

Tone: Let me show you what I built and why I made the choices I did.

Timing Overview

Section	Time	What You're Showing
Opening	2 min	Business problem, HRRP penalties, core question
Phase 1: Data Profiling	6 min	Data shape, target imbalance, missing data, duplicates
Phase 2: EDA	8 min	Correlations, disposition, HbA1c, meds, stats tests
Phase 3: Pipeline	10 min	8 cleaning steps, 7 features, sklearn pipeline
Bridge: Modeling Setup	4 min	CRISP-DM, stakeholders, metrics, imbalance strategy
Wrap-Up + Questions	10 min	Summary, what's next, open discussion

Opening

Start at code cell 0

So the business problem is hospital readmissions. Under the CMS Hospital Readmissions Reduction Program, hospitals get penalized up to 3% of all their Medicare payments if their 30-day readmission rates are too high. In 2024, about 80% of evaluated hospitals got hit with some level of penalty -- over \$560 million industry-wide. That's not a small number.

The question I'm trying to answer is: given the clinical and medication data that's available at the time a diabetic patient is discharged, can we predict which ones are coming back within 30 days? If we can, the hospital can target their expensive follow-up resources -- discharge coaching, medication reconciliation, follow-up calls -- on the patients who actually need them instead of applying them to everyone or applying them to no one.

Phase 1: What Does the Data Look Like?

Ref Code Cell 3 output (data loading)

The dataset is from the UCI repository -- about 102,000 inpatient encounters across 130 US hospitals over 10 years. 50 features covering demographics, diagnoses, medications, lab results, and prior utilization.

Ref Code Cell 5 (target distribution chart)

Here's the target. About 11% of patients are readmitted within 30 days. That's an 8-to-1 class imbalance. If I built a model that just predicted 'not readmitted' for everyone, it'd be 89% accurate and completely useless. So accuracy is off the table as a metric -- I'm using ROC-AUC as the primary comparison metric, and recall matters more than precision because a missed readmission is a missed intervention opportunity and a potential penalty.

Ref Code Cell 6 (schema profile)

First thing I did was profile every column. This is where I found the landmines. The dataset uses question marks instead of NaN for missing values -- pandas doesn't catch those as missing unless you replace them first. Weight is 97% missing, completely unusable. Payer code is 40% missing. And the diagnosis columns have about 700 unique ICD-9 codes each -- way too many categories for any model to learn from.

Ref Code Cell 7 (missing data chart)

This chart drove the cleaning decisions. Anything above the red line gets dropped. Anything between the red and orange line needs imputation. And everything below 5% is minor enough that the pipeline imputers will handle it.

Why 50% and 5% thresholds?

These are heuristics, not hard rules. At 50%+ missing, imputation generates more fiction than fact. The 5% line separates features where default imputers are fine from features needing deliberate strategy. For medical_specialty at 49%, I created an 'Unknown' category rather than imputing because the missingness itself might be informative -- it could indicate an ER admission where no admitting specialist was assigned.

Ref Code Cell 11 (duplicate patients chart)

This was a critical finding. About 71,000 unique patients generated 102,000 encounters. Some patients show up three, four, five times. That's a data leakage problem -- if the same patient ends up in both my training and test sets, the model could memorize patient-specific patterns instead of learning generalizable readmission risk factors. So during cleaning I keep only one encounter per patient.

Phase 2: What Patterns Exist?

Ref Code Cell 16 (Phase 2 header)

Phase 2 is where I go from 'what does the data look like' to 'what relationships exist.' I'm looking for features that actually differentiate readmitted patients from non-readmitted patients.

Ref Code Cell 17 (correlation heatmap)

The correlation matrix is honestly a bit underwhelming -- nothing correlates strongly with readmission. The strongest is number of prior inpatient visits, and even that is under 0.10. That's typical for readmission prediction. The signal is in interactions between features, not individual features. Which is exactly why I'm using tree-based models -- they can learn those interactions automatically.

Ref Code Cell 18 (discharge disposition chart)

Discharge disposition turned out to be one of the most informative features. Patients discharged against medical advice -- AMA -- have a noticeably higher readmission rate. Patients sent to home health versus sent home with self-care have different profiles. And this is also where I identified the expired patients -- codes 11, 19, 20, 21. Dead patients can't be readmitted, so they have to be removed or they artificially inflate the negative class.

Ref Code Cell 20 (HbA1c analysis)

This is probably the most interesting clinical finding, and it replicates the original Strack paper from 2014. It's not about whether a patient's HbA1c is high or normal -- it's about whether it was measured at all. Patients whose HbA1c was measured during the stay have a lower readmission rate regardless of the result. The act of measuring is a proxy for better diabetes management protocols. So I engineered a binary feature called `has_A1c_measured` that captures exactly this.

Ref Code Cell 23-27 (statistical summary table)

I ran five statistical tests. Chi-square on discharge disposition and HbA1c measurement, Mann-Whitney on prior inpatient visits, Kruskal-Wallis on length of stay, and point-biserial correlations on all the numerics. With 100,000 rows, everything is significant -- every p-value is essentially zero. So I'm reporting effect sizes alongside every test. Most effects are small, which is honest and expected. The Mann-Whitney on prior inpatient visits has the largest effect size, confirming what the EDA showed visually.

Why these five specific tests?

The test choice follows from the data types being compared:

- Categorical vs. categorical = Chi-Square (Tests 1-2)
- Numeric (skewed) vs. 2 groups = Mann-Whitney U (Test 3)
- Numeric (skewed) vs. 3+ groups = Kruskal-Wallis (Test 4)
- Numeric vs. binary = Point-Biserial (Test 5)

I didn't pick tests and find variables. I looked at the important relationships from the EDA and picked the test that matched each pair's data types.

Why Mann-Whitney U?

It asks whether two groups come from different distributions, without assuming normality. Take every readmitted patient and count how many non-readmitted patients they outrank on prior inpatient visits. If readmitted patients consistently rank higher, U is large. We used it instead of a t-test because prior inpatient visits is heavily right-skewed and zero-inflated -- most patients have 0 visits. Mann-Whitney works on ranks, so the skew doesn't matter.

Ref Code Cell 28 (EDA Summary)

This is the synthesis. I ranked the predictors by evidence strength, listed four modeling hypotheses, and documented the cleaning requirements that Phase 2 uncovered. This cell is the bridge -- everything above it is exploration, everything below it is engineering.

Phase 3: Building the Pipeline

Ref Code Cell 30 (Phase 3 header)

Phase 3 is where I take everything I learned and turn it into a reproducible machine learning pipeline. Eight cleaning steps, seven engineered features, and a scikit-learn ColumnTransformer that takes raw data in and produces a model-ready matrix out.

Scroll through Code Cells 85-106

The cleaning steps are in a specific dependency order. Question marks get replaced with NaN first -- that has to happen before anything else because all the null-handling logic downstream depends on it. Then I binarize the target, remove expired patients, deduplicate by patient. Each step has a rationale explaining why it exists and why it's in that position.

Ref Code Cell 100 (ICD-9 grouping)

The ICD-9 grouping is one of the more important decisions. 700 unique codes per diagnosis column is way too many for any model. I group them into 9 clinical categories following the methodology from the original Strack paper -- Circulatory, Respiratory, Diabetes, Digestive, and so on. That's a 98% reduction in cardinality while keeping the distinctions that actually matter clinically.

Ref Code Cell 106 (ID columns to string)

This is a subtle one that's easy to miss. The admission type, discharge disposition, and admission source columns look like numbers -- 1, 2, 3, 6 -- but they're actually categorical codes. Code 6 means 'Home Health,' code 2 means 'Short-term Hospital.' If I leave them as integers, the scaler treats them as continuous and the model thinks discharge to Home Health is three times discharge to Short-term Hospital. Converting to string forces the pipeline to one-hot encode them correctly.

How did I know those were categorical?

Three layers. First, the dataset ships with an IDs_mapping.csv file that documents what each code means. Second, anything ending in '_id' in a healthcare dataset is almost always a foreign key to a lookup table -- it's a database naming convention. Third, these are standard UB-04 billing form fields that every US hospital submits. Admission type, discharge disposition, and admission source have been coded this way since the 1980s.

Ref Code Cell 109 (feature engineering)

Seven new features, each with a clinical rationale. The most important one is has_A1c_measured -- the Strack finding we saw in Phase 2. medication_change_count replaces the need to individually model all 23 medication columns by counting how many had dosage changes. total_visits_prior_year combines inpatient, outpatient, and emergency visits into a single utilization index. And polypharmacy flags patients on more than 15 medications -- that threshold comes from Masnoon et al.'s 2017 systematic review of polypharmacy definitions.

Ref Code Cell 112 (drop low-variance meds)

After computing medication_change_count, I drop 21 of the 23 medication columns. Most of them are over 95% 'No' -- near zero variance. Keeping insulin and metformin individually because they

have meaningful variance, and everything else is captured by the composite count. This takes us from what would have been 92 one-hot encoded features down to about 6.

Ref Code Cell 120 (pipeline) and cell 123 (train/test split)

The pipeline is a ColumnTransformer with three branches. Numeric features get median imputation and standard scaling. Categoricals get mode imputation and one-hot encoding. Binary engineered features pass through untouched. The whole thing chains with the classifier in Phase 4, so the entire flow from raw features to predictions is one reproducible object.

Train-test split is 80/20, stratified on the target so the 11% positive rate is preserved in both sets. The test set is locked -- I don't touch it until Phase 6.

Ref Code Cell 126 (pipeline validation)

The validation step confirms the pipeline transforms without errors and produces zero NaN in the output. You can see the feature count after one-hot encoding -- the 16 categorical features expand to about 100+ binary columns, so the total feature space is well over 100 features.

Bridge: Setting Up for Modeling

Ref Code Cells 129-134

Before I start modeling, I wanted to explicitly document a few things that are easy to take for granted.

Cell 129 (CRISP-DM)

The CRISP-DM mapping shows where each notebook phase fits in the methodology. We've completed Business Understanding through Data Preparation. Modeling is next.

Cell 130 (stakeholders)

The stakeholders section explains who actually uses this model. The VP of Case Management is the primary consumer -- they use patient-level risk scores to decide who gets intensive follow-up. The CFO cares about the dollar impact. The CMO sets policy based on the aggregate findings. Hospitalists receive discharge-time alerts with the specific risk factors for each patient.

Cell 131 (success metrics)

I set explicit targets before modeling so there's no moving the goalposts. ROC-AUC above 0.65 is the bar -- the published benchmark for this dataset is 0.64 to 0.67, so exceeding 0.65 confirms the pipeline adds value. Recall above 0.50 means we catch at least half of readmissions. And in Phase 6 I'll compute a custom cost-adjusted net benefit that puts actual dollars on it.

Cell 132 (imbalance strategy)

Three approaches to handle the 8-to-1 class imbalance. Class weights, SMOTE, and XGBoost's `scale_pos_weight`. I'll compare all three in Phase 4. The key thing with SMOTE is it has to happen inside the cross-validation folds, not before the split, or you get data leakage.

Cell 133 (flowchart)

And the conceptual framework shows the end-to-end pipeline with validation gates between each phase. The point is that no phase starts until the previous one has been validated.

Wrap-Up (2 min)

So that's where we are. 135 cells, 15 figures, five statistical tests, a reproducible sklearn pipeline, and a clean 70,000-patient dataset ready for modeling. The next step is Phase 4 -- training four baseline models with logistic regression, random forest, XGBoost, and SVM, comparing them on 5-fold cross-validation, then tuning the best performers in Phase 5.

The thing I'm most confident about is the feature engineering -- the prior utilization composite and the HbA1c measurement flag are grounded in the EDA and the published literature. The thing I'm watching for is whether the interactions we saw in the Phase 2 heatmaps actually surface in the tree-based model importance rankings. If `number_inpatient` and `has_A1c_measured` show up in the top 10 of both the logistic regression and the XGBoost, that's convergence across methods and it strengthens whatever recommendation comes out of Phase 6.

Anticipated Questions & Answers

Prepared responses for likely peer questions during the discussion.

Why not use all 3 classes instead of binarizing?

The HRRP penalty is specifically about 30-day readmissions. A patient readmitted at day 45 doesn't trigger the penalty. Binarizing to `<30` vs. everything else directly aligns the model with the business problem.

Why keep the longest stay instead of the first or last?

The longest stay has the most clinical information -- more procedures, more medication decisions, more diagnostic data. It gives the model the richest feature set to learn from.

Isn't 0.65 AUC kind of low?

For readmission prediction, that's typical. The reason is social determinants -- housing stability, transportation, caregiver support -- are among the strongest predictors of readmission and they're not in clinical data. An AUC of 0.65 that costs \$500 per intervention and saves \$13,000 per avoided penalty is still net-positive for the hospital.

Why four models instead of just XGBoost?

The rubric requires at least three. But the real reason is that logistic regression gives you interpretable coefficients and odds ratios that clinicians trust, while XGBoost captures non-linear interactions. If a feature ranks high in both, the recommendation is stronger than if it only shows up in one. SVM adds a completely different learning paradigm for diversity.

What's your biggest concern going into modeling?

The class imbalance. With 11% positive, the default threshold of 0.5 will almost certainly underpredict readmissions. I expect threshold optimization in Phase 6 to matter more than model selection in Phase 5.

Why Mann-Whitney U?

It asks whether two groups come from different distributions without assuming normality. Take every readmitted patient and count how many non-readmitted patients they outrank on prior inpatient visits. If readmitted patients consistently rank higher, U is large. We used it instead of a t-test because prior inpatient visits is heavily right-skewed and zero-inflated. Mann-Whitney works on ranks, so the skew doesn't matter.

Why these five specific tests?

The test follows from the data types. Categorical vs. categorical = Chi-Square. Numeric (skewed) vs. 2 groups = Mann-Whitney. Numeric (skewed) vs. 3+ groups = Kruskal-Wallis. Numeric vs. binary = Point-Biserial. I looked at the important relationships from the EDA and picked the test that matched each pair's data types. The test follows from the data, not the other way around.

Feature Type	Target Type	Correct Test
Categorical	Categorical	Chi-Square
Continuous (skewed)	2 groups	Mann-Whitney U
Continuous (skewed)	3+ groups	Kruskal-Wallis
Continuous	Binary	Point-Biserial Correlation

How did you know the ID columns were categorical?

Three layers. First, the dataset ships with an IDs_mapping.csv file that documents what each code means. Second, anything ending in '_id' in a healthcare dataset is almost always a foreign key to a lookup table -- it's a database naming convention. Third, these are standard UB-04 billing form fields that every US hospital submits to payers. Admission type, discharge disposition, and admission source have been coded this way since the 1980s.

What does the 95% CI tell us on the diagnosis count chart?

It shows how much to trust each data point. For diagnoses 1-8, the band is tight -- thousands of patients per bucket, so the upward trend is reliable. Above 9-10 diagnoses, the band explodes because very few patients have that many diagnoses. The spike at 11 is probably a handful of patients. The model sees the raw number and handles sparse regions through regularization -- we don't need to remove the outliers.